



9
3
4.5 sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

الواجهات (Interface)

د. ماهر صارم



RB Informatcs; 30/5/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

ما هو الـ Interface؟

هو مخطط (Blueprint) يحدد ما يجب أن يفعله الكلاس دون أن يحدد كيف سينفذه، يحتوي على توقعات التوابع فقط (اسم التابع، Parameters، ونوع القيمة المعادة) دون كود تنفيذي وأي كلاس يقوم بتطبيق (implements) هذه الواجهة يكون ملزماً بكتابة الكود التنفيذي لجميع توابعه.

مقارنة بين الـ abstract والـ interface:

Abstract class	Interface
يمكن للكلاس أن يرث (extends) من كلاس تجريدي واحد فقط.	يمكن للكلاس تضمين (implements) أكثر من واجهة بنفس الوقت.
يمكن أن تحوي على توابع مجردة (دون كود تشغيلي) وتوابع عادية (مع كود تشغيلي).	جميع التوابع فيه تكون مجردة (abstract) تكتب بلا كود تشغيلي.
يمكن أن تحوي على متغيرات لجميع أنواع الصلاحيات (private, public, ...).	أي متغير فيها يكون تلقائياً ثابت ولا يمكن تعديله (public static final).



مثال توضيحي لـ interface بسيط:

```
public interface Animal {
    void move();
    void eat();
}
```

شرح الكود:

public interface Animal

إنشاء Interface اسمه Animal

```
public class Fish implements Animal{
    @Override
    public void move(){
        System.out.println("the fish swims");
    }
    @Override
    public void eat(){
        System.out.println("the fish eats sea
plants");
    }
}
```

void move(); / void
eat()

أي حيوان يجب أن يمتلك
دالة للحركة ودالة للأكل

implements
Animal

```
public class Lion implements Animal{
    @Override
    public void move(){
        System.out.println("the lion runs");
    }
    @Override
    public void eat(){
        System.out.println("the lion eats meat");
    }
}
```

تعني أن الكلاسان Lion
و Fish سيطبقان جميع
الدوال الموجودة داخل الـ
Interface Animal كل
بطريقته.

Override@

```
import java.util.ArrayList;
public class Main {
    public static void main (String[] args)
    {
        ArrayList<Animal>animals=new ArrayList<>();
        Animal lion=new Lion();
        Animal fish=new Fish();
        animals.add(lion);
        animals.add(fish);
        for(Animal animal:animals){
            animal.move();
            animal.eat();
        }
    }
}
```

تعني أننا نعيد كتابة
(implement) الدالة
الموجودة في الـ Interface
"وقد ذكرنا سابقاً أن وجودها
ليس إلزامياً لكنها تساعد
المترجم على اكتشاف
الأخطاء إذا كتبنا اسم الدالة
بشكل خاطئ مثلاً"

▪ في الـ main:

أنشأنا ArrayList يمكنها تخزين أي كائن يُطبّق الـ Interface Animal لذلك نستطيع وضع:

Lion

Fish

وأي حيوان جديد يطبق Animal

▪ `Animal lion = new Lion _ ;`
`Animal fish = new Fish _ ;`

نلاحظ أننا استخدمنا مرجع Animal وليس Lion أو Fish وهذه إحدى فوائد الـ Interface لأنها تجعل الكود أكثر مرونة.

▪ `animals.add(fish) _ ;`
`animals.add(lion)`

دالة تقوم بإضافة عناصر لـ الـ ArrayList

▪ `for (Animal animal : animals)`

enhance for loop هي for-each

وبذلك بعد استدعاء الدوال

`animal.move`

`animal.eat`

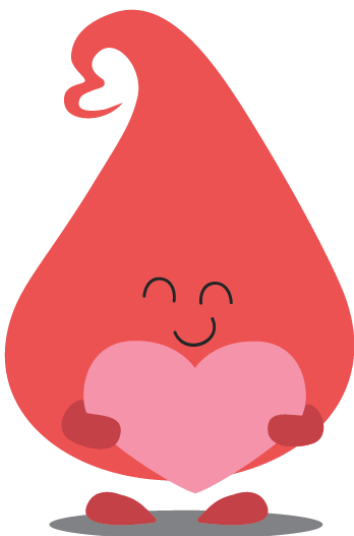
داخل الـ for-each ستكون النتيجة:

The Lion runs

The Lion eats meat

The Fish swims

The Fish eats sea plants



انتهت المحاضرة التاسعة وانتهى المقرر 😊 بالتوفيق!